

Université Ibn Zohr
École Supérieure de Technologie – Agadir
Filière : Conception et Développement Logiciel
Année universitaire : 2025-2026

Rapport de SFE

Conception et développement d'une plateforme interne de gestion
et d'encadrement des stagiaires
Application : StageFlow

Réalisé par : Reda Bouzid
Date de soutenance : 28/05/2026
Encadrant : Neuman Charhbili

Table des matières

Remerciements	4
Résumé	5
Introduction générale	6
1 Présentation de l'organisme d'accueil	7
1.1 Présentation générale	7
1.2 Pourquoi Vala ?	7
1.3 Services proposés	7
1.4 Carrière et ressources humaines	8
1.4.1 Fiche de la société	8
1.5 Organigramme de l'entreprise	8
2 Présentation du projet	10
2.1 Contexte général et justification	10
2.2 Objectifs du projet	10
2.3 Périmètre fonctionnel	11
3 Analyse fonctionnelle détaillée	12
3.1 Identification des acteurs et responsabilités	12
3.2 Besoins fonctionnels	12
3.2.1 Gestion de l'authentification et des comptes	12
3.2.2 Gestion des stages et des comptes étudiants	13
3.2.3 Gestion de l'encadrement et du suivi des tâches	13
3.2.4 Gestion des rapports de stage	13
3.3 Diagrammes UML	13
3.3.1 Diagramme des cas d'utilisation	13
4 Parcours utilisateurs critiques	15
4.1 Parcours étudiant	15
4.2 Parcours superviseur	15
5 Modèle de données PostgreSQL	18
5.1 Conception de la base de données	18
5.2 Diagramme de classes UML et relations entités	18
6 Architecture technique	21
6.1 Architecture globale	21
6.2 Stack technologique	21
6.3 Sécurité et validation	21
6.3.1 Flux complet d'authentification JWT	22

6.4	Fiabilité et qualité	23
7	Interfaces utilisateur et captures d'écran	24
7.1	Authentification — Écran de connexion	24
7.2	Tableau de bord administrateur	25
7.3	Gestion des stagiaires	25
7.4	Création de stage	25
7.5	Gestion des tâches	26
7.6	Suivi de progression	26
7.7	Espace étudiant	27
7.8	Gestion des rapports	27
7.9	Workflow d'encadrement (vue synthétique)	28
8	Outils et technologies utilisés	29
8.1	Git et GitHub	29
8.2	Node.js	29
8.3	Express.js	30
8.4	PostgreSQL	30
8.5	Docker	30
8.6	JWT (JSON Web Token)	31
	Conclusion générale et perspectives	32

Table des figures

1.1	Logo de l'organisme d'accueil	7
1.2	Organigramme et fonctions clés	9
3.1	Diagramme des cas d'utilisation : interactions entre Administrateur, Superviseur et Étudiant	14
4.1	Figure IV.1 : : Diagramme de séquence : création du compte étudiant et suivi de stage	16
4.2	Figure IV.2 : : Diagramme d'activités : avancement et suivi des tâches entre étudiant et superviseur	17
5.1	Diagramme de classes UML : entités principales et relations du modèle de données	19
6.1	Figure IV.3 : : Diagramme de séquence du flux d'authentification JWT	22
7.1	Authentification — Écran de connexion (image fournie ultérieurement)	24
7.2	Tableau de bord Administrateur (image fournie ultérieurement)	25
7.3	Gestion des stagiaires — liste et fiche (image fournie ultérieurement)	25
7.4	Création de stage — formulaire de saisie (image fournie ultérieurement)	26
7.5	Gestion des tâches — création et suivi (image fournie ultérieurement)	26
7.6	Suivi de progression — indicateurs et timeline (image fournie ultérieurement) . .	26
7.7	Espace étudiant — tableau personnel et soumission de livrables (image fournie ultérieurement)	27
7.8	Gestion des rapports — soumission et workflow de revue (image fournie ultérieurement)	27
7.9	Workflow d'encadrement — synthèse des interactions (image fournie ultérieurement)	28
8.1	Git et GitHub — contrôle de version distribué	29
8.2	Node.js — runtime JavaScript côté serveur	29
8.3	Express.js — framework web minimaliste	30
8.4	PostgreSQL — système de gestion de base de données relationnelle	30
8.5	Docker — conteneurisation et portabilité	30
8.6	JWT — authentification par jeton stateless	31

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant, Neuman Charhbili, pour son accompagnement rigoureux, ses orientations méthodologiques et sa disponibilité constante tout au long de ce projet de fin d'études. Je remercie également l'ensemble du corps enseignant de l'École Supérieure de Technologie – Agadir pour la qualité de la formation dispensée, qui m'a permis d'acquérir des bases solides en ingénierie logicielle, en architecture des systèmes et en gestion de projet. Mes remerciements vont aussi à ma famille et à mes proches pour leur soutien moral, leur patience et leur encouragement durant toute la durée de ce travail.

Résumé

StageFlow est une plateforme interne de gestion et de suivi des stagiaires conçue pour structurer et automatiser le processus d'encadrement académique et professionnel au sein d'une institution. La plateforme centralise l'ensemble du cycle de vie du stage : création du stage par le superviseur, génération du compte étudiant, suivi des tâches assignées, encadrement continu par remarques et validation des rapports finaux.

L'application adopte une architecture full-stack moderne avec un backend Node.js/Express sécurisé par JWT et RBAC, une validation stricte des données via Joi, et un frontend React/Vite offrant des interfaces adaptées aux trois rôles : Administrateur, Superviseur et Étudiant. La persistance des données est assurée par PostgreSQL avec un modèle relationnel robuste et auditable.

Le système opère selon un flux clair : après une acceptation externe du stage, le superviseur crée le compte étudiant et envoie automatiquement une invitation d'activation par courriel. L'étudiant accède ensuite à la plateforme pour consulter ses tâches, suivre son avancement et soumettre son rapport. Toutes les interactions sont historisées pour assurer la traçabilité complète et l'auditabilité des décisions.

Mots-clés : gestion interne des stagiaires, plateforme d'encadrement, trois rôles, architecture MVC, JWT, RBAC, React, Node.js, PostgreSQL, supervision académique.

Introduction générale

La gestion des stagiaires au sein d'une institution ou d'une structure d'accueil requiert une coordination rigoureuse entre plusieurs acteurs : superviseurs de stage, administrateurs et étudiants. Lorsque ces activités demeurent fragmentées (courriels, feuilles de calcul, dossiers isolés), le suivi devient inefficace, les décisions manquent de traçabilité et l'encadrement pédagogique s'en trouve compromis.

Le projet StageFlow répond à cette problématique en proposant une plateforme interne de gestion des stagiaires, spécifiquement conçue pour centraliser et structurer le processus d'encadrement. Son objectif n'est pas d'automatiser le recrutement ou la sélection des candidats (qui restent des processus externes ou manuels), mais de fournir un cadre fiable pour la gestion opérationnelle des stages une fois les étudiants acceptés : création et pilotage des stages, génération des comptes étudiants, assignation des tâches, suivi continu de la progression, collecte des remarques pédagogiques et validation des rapports.

Cette approche améliore la communication entre superviseurs et étudiants, garantit la traçabilité complète des décisions et actions, et renforce la cohérence de l'encadrement académique. La plateforme s'appuie sur une architecture full-stack moderne avec une authentification sécurisée (JWT), un contrôle d'accès par rôles (RBAC) et une validation stricte des données (Joi).

Ce rapport présente successivement la vision du projet, l'analyse fonctionnelle détaillée, les parcours utilisateurs critiques, l'architecture technique MVC retenue, puis les dispositifs de sécurité, fiabilité et qualité implémentés pour garantir la robustesse et l'auditabilité de l'application.

Chapitre 1

Présentation de l'organisme d'accueil

1.1 Présentation générale

L'organisme d'accueil présenté dans ce rapport est Vala (également mentionné localement comme "Vala Bleu"). Fondée en 2007, la société a son siège social à Agadir (région Souss-Massa) et dispose d'une présence commerciale à l'international, notamment à Londres (Royaume-Uni). Vala s'est spécialisée dans les services d'hébergement web et d'enregistrement de noms de domaine, et opère principalement sur les marchés européen, américain et nord-africain.

Les valeurs professionnelles qui guident l'entreprise sont les suivantes :

- Confiance, engagement et transparence ;
- Sérieux, performance et proactivité ;
- Continuité, disponibilité et qualité de service.



FIGURE 1.1 – Logo de l'organisme d'accueil

1.2 Pourquoi Vala ?

Vala place la satisfaction client au cœur de sa stratégie. L'entreprise propose des solutions d'hébergement et des services cloud conçus pour garantir la continuité des activités en ligne de ses clients. Son infrastructure est pensée pour offrir fiabilité, simplicité d'usage et compétitivité économique, avec un support technique assuré 24h/24 et 7j/7 par une équipe d'experts. La qualité, la fiabilité et l'attention au détail constituent les priorités opérationnelles de Vala.

1.3 Services proposés

Parmi les services proposés figurent :

- Hébergement web à haute performance ;
- Serveurs rapides et sécurisés ;
- Messagerie professionnelle ;

- Solutions de création de sites sans codage ;
- Développement de sites e-commerce évolutifs ;
- Migration gratuite de services ;
- Sécurité avancée des infrastructures ;
- Choix de data centers proches des clients cibles.

1.4 Carrière et ressources humaines

Vala favorise l'apprentissage continu et le développement professionnel. L'entreprise offre des opportunités d'évolution dans un environnement dynamique et technologique. Elle recrute des profils motivés, capables de s'adapter rapidement à des contraintes techniques et organisationnelles.

1.4.1 Fiche de la société

Date de création	30/09/2020
Nom de la société	Creative Internet Solutions
Forme juridique	S.A.R.L
Activités	Hébergement et création de site web
Secteur	Production digitale
Adresse	Immeuble Safwa, Boulevard Hassan 1er, 2e étage N°6, Dakhla – Agadir 80000, Maroc
Téléphone	00 212 528 21 3045 / 00 212 528 22 5522
Effectif	11
Directeur général	M. Kayouh Abdelbast

TABLE 1.1 – Fiche d'information de la société Vala

1.5 Organigramme de l'entreprise

Les responsabilités clés communiquées sont les suivantes :

- Directeur Général : M. Abdelbast Kayouh ;
- Directeur Projet Web : M. Khalid Jbour ;
- Responsable Commercial : M. Abdelbast Kayouh ;
- Attachée à la Direction : Mme Asma El Boudi ;
- Service Comptabilité et Gestion des RH : Mme Asma El Boudi ;
- Service Après-Vente : Mme Wafa Fouks ;
- Développeur : M. Neuman Charhbili ;
- Responsable Service Clientèle : Mme Hanane Bayen ;
- Développeur : M. Bachir El Omari ;
- Gestion de la Relation Client : Mme Karima Chaoui ;
- Service du Personnel : Mme Soukaina Ouahmid, Mme Fatiha Disky.

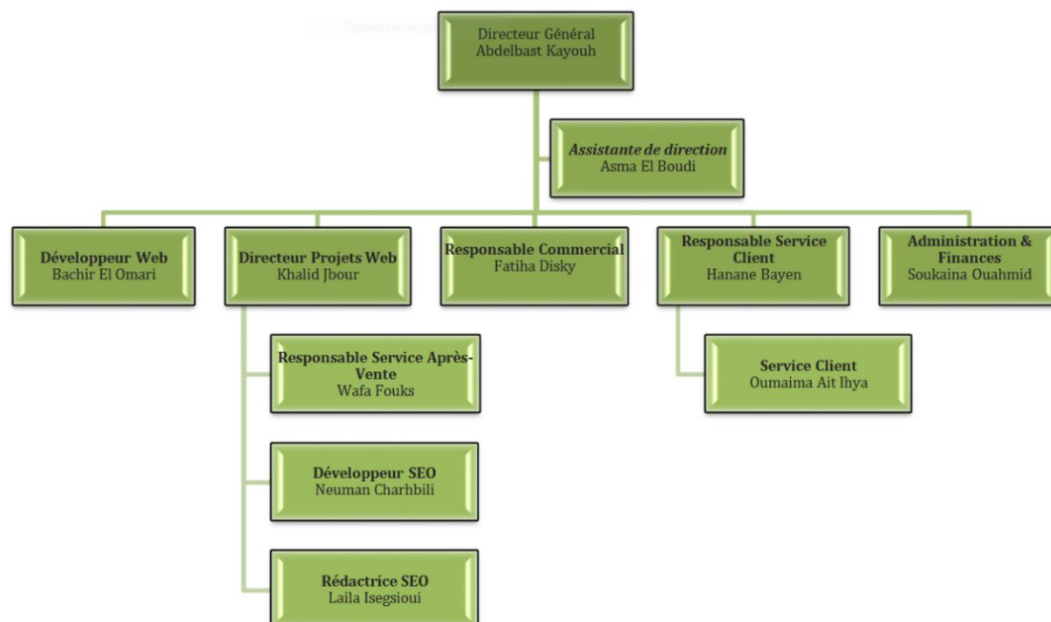


FIGURE 1.2 – Organigramme et fonctions clés

Chapitre 2

Présentation du projet

2.1 Contexte général et justification

Dans de nombreuses institutions, la gestion des stagiaires souffre d'une absence d'intégration entre les outils utilisés par les superviseurs, les administrateurs et les étudiants. Cette dispersion de l'information crée des ruptures opérationnelles : informations perdues entre courriels, validations tardives, historique incomplet des décisions et difficulté à mesurer l'avancement réel des missions et du travail accompli. Ces limites affectent à la fois l'efficacité opérationnelle de l'institution et la qualité de l'expérience vécue par l'étudiant stagiaire.

La justification de StageFlow repose sur la nécessité d'un système unifié et interne, capable d'orchestrer de manière fiable et auditable les différentes phases du suivi d'un stage : enregistrement du stage, affectation et suivi des tâches, encadrement continu, collecte des remarques pédagogiques et validation finale des rapports. En centralisant les données et en clarifiant les responsabilités par rôle, la plateforme réduit les ambiguïtés, améliore la communication entre superviseurs et étudiants, et renforce la gouvernance et la traçabilité du processus.

2.2 Objectifs du projet

Objectif principal

Concevoir, développer et déployer une plateforme web full-stack interne dédiée à la gestion et au suivi des stagiaires, permettant aux superviseurs de créer et de piloter les stages, de gérer les comptes étudiants, d'assurer le suivi des tâches assignées, de fournir un encadrement continu et de valider les rapports finaux, tout en garantissant une sécurité robuste, une traçabilité complète et une audibilité des actions.

Objectifs spécifiques

- Fournir un flux opérationnel clair où le superviseur crée le stage et les comptes étudiants après acceptation externe, et reçoit automatiquement une invitation d'activation.
- Automatiser l'envoi des notifications (invitations, rappels) et conserver un historique complet et auditable de toutes les actions et décisions.
- Mettre en place une sécurité robuste par authentification JWT et contrôle d'accès par rôles (RBAC), et valider strictement les données à l'entrée via Joi.
- Mettre à disposition des superviseurs et de l'administrateur des tableaux de bord et des indicateurs pour le suivi pédagogique, la progression des tâches et l'état des rapports.
- Concevoir une architecture modulaire (API REST, MVC) et testable facilitant la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités.

2.3 Périmètre fonctionnel

Le périmètre fonctionnel de StageFlow couvre les besoins cœur du processus de suivi des stagiaires au sein d'une institution, avec une séparation claire entre les responsabilités de chaque acteur et un chaînage logique des opérations. La plateforme prend en charge :

- L'authentification fiable, l'autorisation basée sur les rôles et la gestion des profils utilisateurs.
- L'administration des entités cœur : étudiants/stagiaires, superviseurs et structures d'accueil (sociétés, organisations).
- La création et la gestion des stages par les superviseurs (description, dates, objectifs, encadrement), ainsi que la gestion manuelle des comptes étudiants associés.
- L'assignation et le suivi du travail via des tâches avec statuts de progression et retours d'encadrement du superviseur.
- La gestion documentaire du rapport de stage : rédaction par l'étudiant, soumission, revue par le superviseur et décision finale (validation ou demande de corrections).
- L'audit et la supervision par des indicateurs pour appuyer le pilotage pédagogique et administratif.

Le système n'inclut pas de module de publication d'offres, de recherche d'opportunités ou de candidature autonome par les étudiants (ces processus relèvent d'une gestion externe ou manuelle antérieure à l'entrée dans StageFlow). Les fonctionnalités avancées hors périmètre cœur (intégrations complexes, analytique prédictive, signatures électroniques) ne constituent pas le principal de cette version, mais l'architecture retenue en facilite l'ajout progressif.

Chapitre 3

Analyse fonctionnelle détaillée

3.1 Identification des acteurs et responsabilités

Le système met en interaction trois acteurs principaux, chacun disposant d'un champ de responsabilité clairement défini afin d'éviter les chevauchements décisionnels et assurer une gouvernance efficace.

L'Administrateur assure la gouvernance technique et administrative de la plateforme : gestion des comptes et des permissions, surveillance de la cohérence et de l'intégrité des données, contrôle des accès, consultation des logs et supervision des incidents de sécurité. Les comptes administrateurs sont créés manuellement au niveau de la base de données par les opérateurs du service, garantissant un contrôle strict de ce rôle privilégié.

Le Superviseur est l'acteur opérationnel central. Il s'inscrit librement via l'authentification standard et dispose de l'ensemble des responsabilités d'encadrement : création et gestion des stages (description, dates, objectifs, affectations), création des comptes étudiants nécessaires, assignation des tâches granulaires, suivi de l'avancement, fourniture de remarques pédagogiques et expertise sur le rapport soumis. Lorsqu'un superviseur crée un compte étudiant, le système envoie automatiquement un courriel d'activation permettant à l'étudiant de définir son mot de passe et accéder à la plateforme.

L'Étudiant (ou stagiaire) dispose d'un compte créé par le superviseur et accède à son espace personnel dédié au stage. Il consulte les tâches assignées, met à jour son avancement, consulte les remarques du superviseur et soumet son rapport final via l'interface. Chaque action est historisée pour assurer la traçabilité complète et permettre aux superviseurs et à l'administrateur d'auditer le suivi pédagogique.

Cette répartition des trois rôles soutient une gouvernance responsable et auditable du processus, où chaque action est traçable et reliée à un acteur identifié.

3.2 Besoins fonctionnels

3.2.1 Gestion de l'authentification et des comptes

Le système doit permettre l'authentification, la gestion de session et la récupération de compte de manière fiable et sécurisée. Dans un environnement multi-rôles, la gestion des identités ne se limite pas à la connexion ; elle implique aussi un contrôle fin des permissions selon les responsabilités métier assignées à chaque rôle.

L'application implémente une authentification par jeton JWT (JSON Web Token), accompagnée de mécanismes de validation stricte des entrées, de vérification de l'état du compte et de protection des routes sensibles. La gestion des mots de passe suit des pratiques de sécurité solides : hachage bcrypt, règles de robustesse, possibilité de renouvellement, afin de réduire les risques de compromission des accès.

3.2.2 Gestion des stages et des comptes étudiants

Dans StageFlow, la création et la gestion des stages sont réalisées par les superviseurs. Un stage comporte une description claire, des dates de début et de fin, des objectifs opérationnels et les éléments d'encadrement (superviseur responsable, lieu, structure d'accueil). Le superviseur crée manuellement les comptes étudiants associés aux stages ; chaque création de compte déclenche automatiquement l'envoi d'un courriel d'activation permettant à l'étudiant de définir son mot de passe et accéder à la plateforme.

Le système conserve un historique complet des modifications (qui a créé/modifié quel stage, dates d'action, commentaires), assurant la traçabilité et l'auditabilité. Le système ne gère pas de module de publication d'offres ni de candidatures autonomes : l'admission au stage et la création du compte étudiant relèvent d'un processus externe antérieur ou d'un traitement manuel par le superviseur basé sur des critères de sélection hors plateforme.

3.2.3 Gestion de l'encadrement et du suivi des tâches

Une fois le compte étudiant créé et le stage enregistré, la plateforme structure le travail opérationnel par des tâches. Le superviseur décompose la mission en tâches granulaires, fixe des échéances et observe l'évolution des statuts (créée, en cours, terminée, retour attendu, etc.). L'étudiant accède à son espace personnel pour mettre à jour l'avancement de ses tâches, signaler des blocages et recevoir des remarques exploitables du superviseur.

Le système maintient un historique complet des interactions : commentaires, changements de statut, dates de mise à jour, auteur de chaque action. Cette traçabilité soutient l'évaluation continue et permet une capitalisation des retours d'expérience en fin de cycle.

3.2.4 Gestion des rapports de stage

La gestion des rapports constitue la phase de clôture du stage. L'étudiant rédige et soumet son document final via l'interface (format texte, fichier ou autre selon la plateforme). Le superviseur dispose d'un workflow de revue lui permettant d'examiner le rapport, de l'accepter ou de demander des corrections et des révisions, assurant la qualité pédagogique et professionnelle des livrables.

Le système garantit l'intégrité de ce cycle en empêchant les validations incohérentes, en enregistrant précisément l'auteur de chaque décision, la date de traitement et les remarques associées. Cette traçabilité est essentielle pour sécuriser l'évaluation académique et professionnelle, et pour constituer une archive fiable et auditable des livrables de stage.

3.3 Diagrammes UML

Cette section rassemble les diagrammes UML utilisés pour documenter la modélisation et les interactions de la plateforme StageFlow. Les diagrammes fournissent une vue fonctionnelle et structurelle du système, en se concentrant sur les trois rôles implémentés (Administrateur, Superviseur, Étudiant) et leurs interactions avec les services exposés par l'application.

3.3.1 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation ci-dessous synthétise les principaux scénarios fonctionnels pris en charge par StageFlow : création et gestion des stages, création des comptes étudiants par le superviseur, suivi des tâches, encadrement et validation des rapports. Il illustre les services métier offerts par la plateforme sans évoquer de fonctions externes de recrutement ou de publication d'offres.

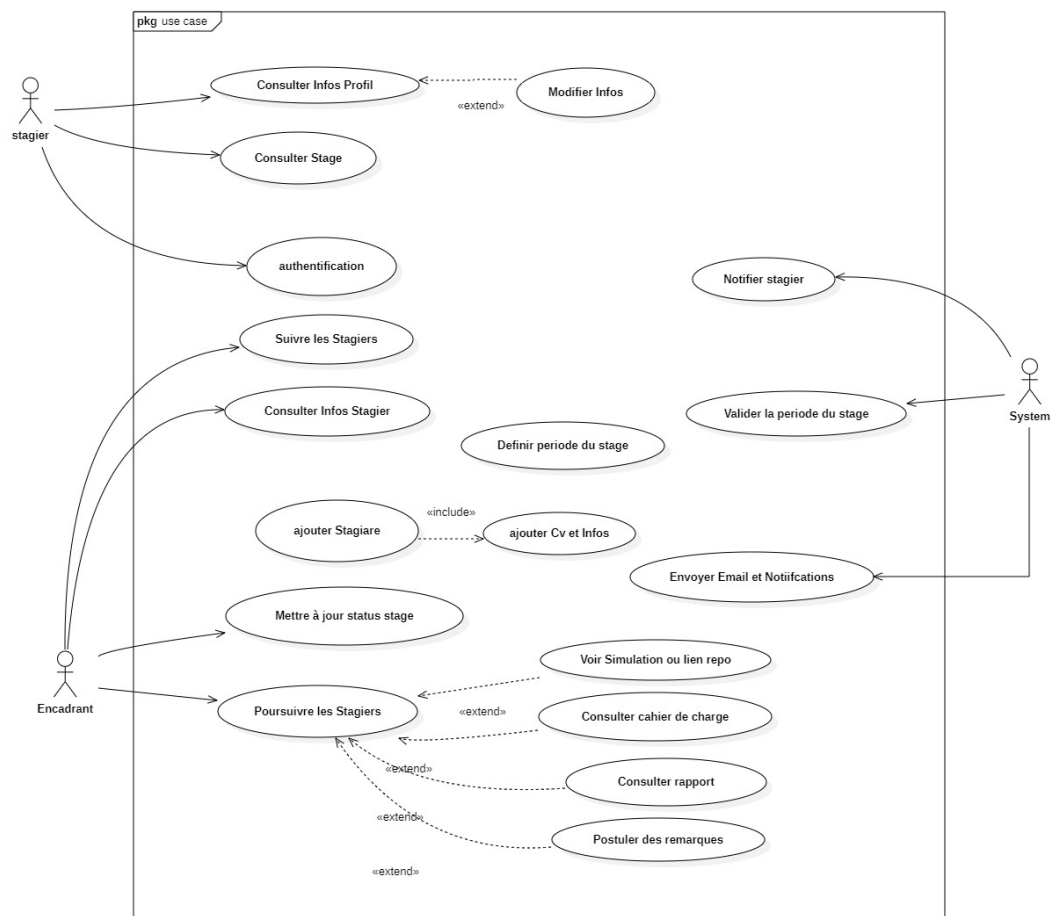


FIGURE 3.1 – Diagramme des cas d'utilisation : interactions entre Administrateur, Superviseur et Étudiant

Chapitre 4

Parcours utilisateurs critiques

4.1 Parcours étudiant

Le parcours étudiant débute après la création de son compte par un superviseur. L'étudiant reçoit un courriel d'activation contenant un lien de réinitialisation de mot de passe. Après définition de son mot de passe, il accède à son espace personnel de la plateforme.

Une fois authentifié, l'étudiant consulte le stage auquel il est rattaché, les objectifs définis par le superviseur, et l'ensemble des tâches qui lui ont été assignées. Il met régulièrement à jour le statut de chaque tâche pour refléter sa progression réelle, et consulte les remarques pédagogiques et conseils fournis par le superviseur.

Le parcours s'achève par la rédaction et la soumission du rapport final. L'étudiant peut suivre l'état de son rapport au sein de la plateforme (en attente de revue, corrections demandées, approuvé, etc.). Toutes les actions de l'étudiant sont historisées afin de permettre une auditabilité complète et un suivi pédagogique documenté.

4.2 Parcours superviseur

Le parcours superviseur s'amorce dès son inscription et la création d'un stage. Le superviseur s'inscrit librement via l'authentification standard de la plateforme. Il crée ensuite un stage en fournissant la description, les dates, les objectifs pédagogiques et les références de la structure d'accueil.

Une fois le stage enregistré, le superviseur crée les comptes étudiants associés. Chaque création de compte déclenche l'envoi automatique d'un courriel d'activation permettant à l'étudiant de se connecter. Le superviseur définit ensuite le cadre opérationnel : il décompose la mission en tâches planifiées, fixe les échéances, et assigne chaque tâche à l'étudiant.

Tout au long du stage, le superviseur suit l'avancement des tâches, fournit des remarques régulières pour orienter la progression et soutenir l'apprentissage, identifie les écarts éventuels et propose des actions correctives. En fin de cycle, il traite le rapport soumis : il examine le contenu, valide ou demande des révisions en s'appuyant sur des critères explicites et pédagogiques. Sa décision finale garantit une clôture formelle et traçable du stage.

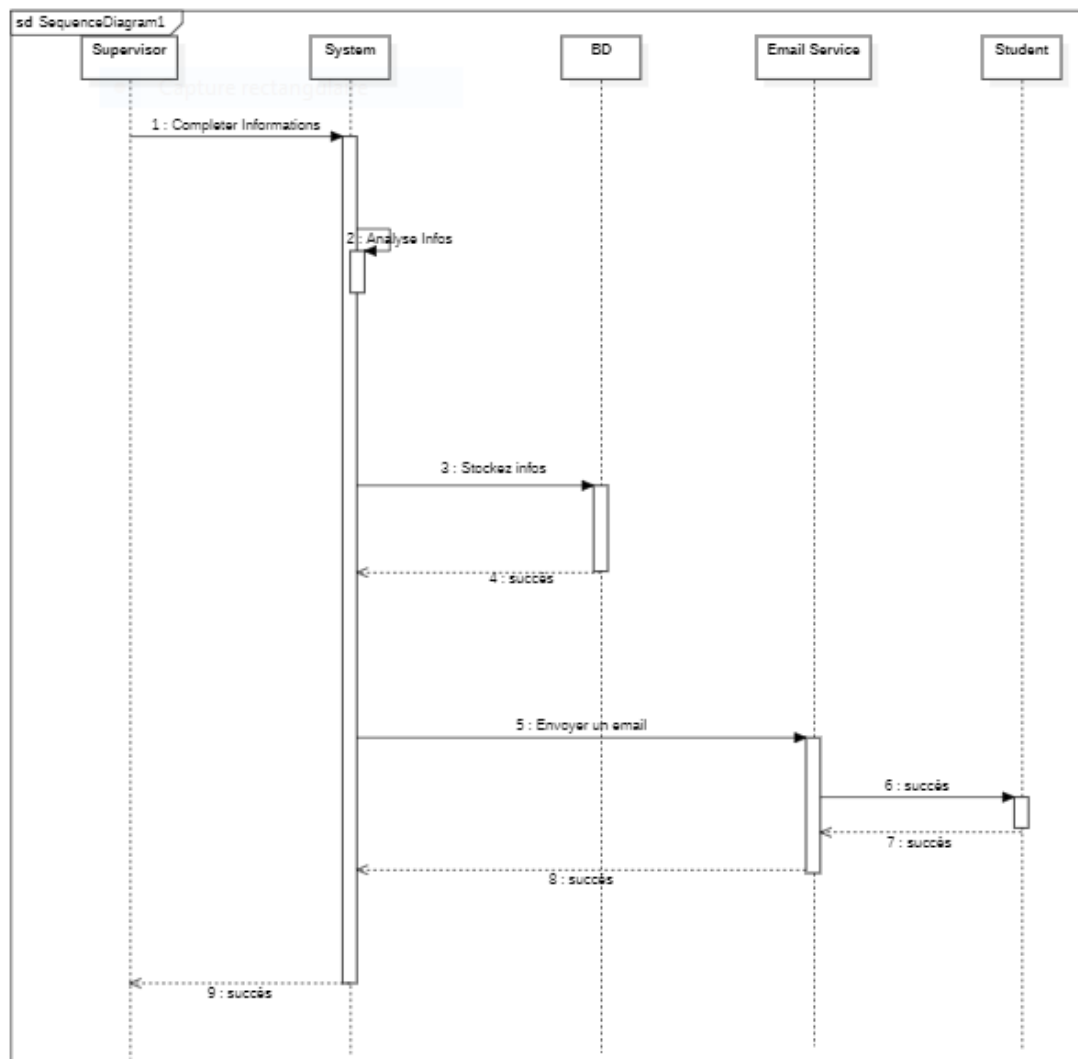


FIGURE 4.1 – Figure IV.1 : : Diagramme de séquence : création du compte étudiant et suivi de stage

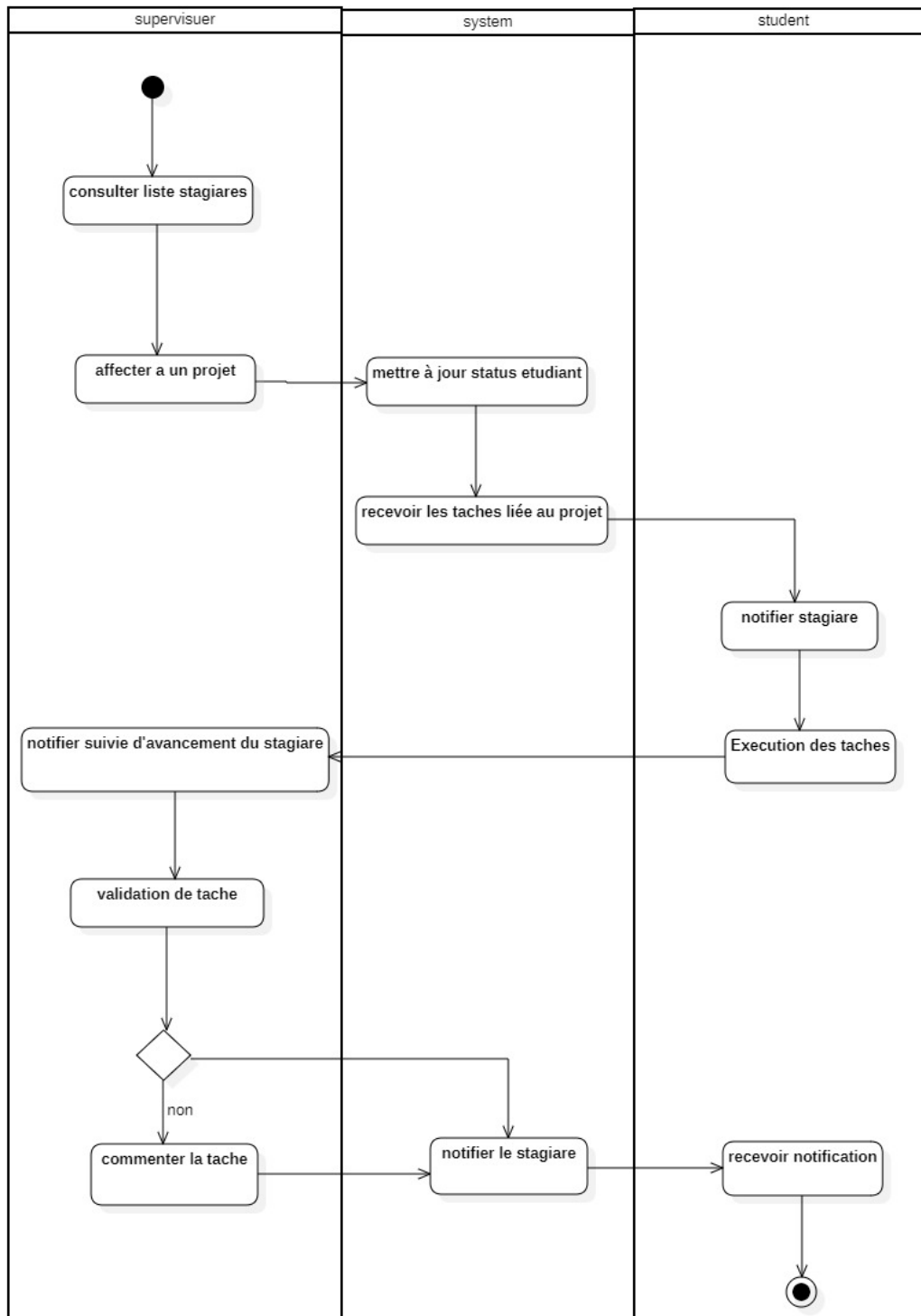


FIGURE 4.2 – Figure IV.2 : : Diagramme d'activités : avancement et suivi des tâches entre étudiant et superviseur

Chapitre 5

Modèle de données PostgreSQL

5.1 Conception de la base de données

La base de données PostgreSQL constitue le fondement de la persistance des données pour la plateforme StageFlow. Elle est conçue selon un modèle relationnel strict qui garantit l'intégrité des données, la cohérence des transactions et la traçabilité complète des opérations métier. Le schéma relationnel reflète fidèlement les entités métier identifiées lors de l'analyse fonctionnelle : utilisateurs, stages, tâches, remarques et rapports.

L'architecture de la base de données s'appuie sur des contraintes d'intégrité référentielle fortes, des index stratégiques pour optimiser les performances des requêtes fréquentes, et des mécanismes d'historisation pour conserver une trace auditable de toutes les modifications critiques. Chaque table est conçue pour supporter les workflows métier spécifiques de StageFlow, tout en facilitant l'évolution future du système.

5.2 Diagramme de classes UML et relations entités

Le diagramme de classes UML ci-dessous présente la structure complète du modèle de données de StageFlow. Il illustre les entités principales, leurs attributs, et les relations qui les unissent, en correspondance directe avec le schéma relationnel implémenté dans PostgreSQL.

Analyse des relations entre entités Les relations entre les entités du modèle de données sont conçues pour assurer la cohérence et l'intégrité du système :

Utilisateurs et Rôles : L'entité **Utilisateur** représente l'ensemble des acteurs du système (administrateurs, superviseurs, étudiants). Elle possède un attribut **role** qui définit le rôle de chaque utilisateur, implémentant ainsi le modèle RBAC (Role-Based Access Control). Cette relation permet de contrôler finement les accès aux fonctionnalités selon les permissions attribuées à chaque rôle.

Projets et Superviseurs : La relation entre **Projet** et **Superviseur** est de type plusieurs-à-un. Un superviseur peut créer et gérer plusieurs projets (stages), mais chaque projet est rattaché à un seul superviseur responsable. Cette relation garantit une clarification des responsabilités et facilite le suivi pédagogique.

Projets et Stagiaires : La relation entre **Projet** et **Stagiaire** est de type plusieurs-à-un. Un projet peut concerner plusieurs stagiaires, mais un stagiaire ne peut être associé qu'à un seul projet à la fois. Cette approche garantit une gestion claire des affectations tout en permettant le suivi de plusieurs stagiaires au sein d'un même projet.

Tâches et Projets : L'entité **Tache** est liée à un projet spécifique. Chaque tâche possède des attributs de suivi (statut, dateDébut, dateFin) qui évoluent selon le workflow métier. Cette relation permet un découpage structuré du travail en unités gérables et traçables.

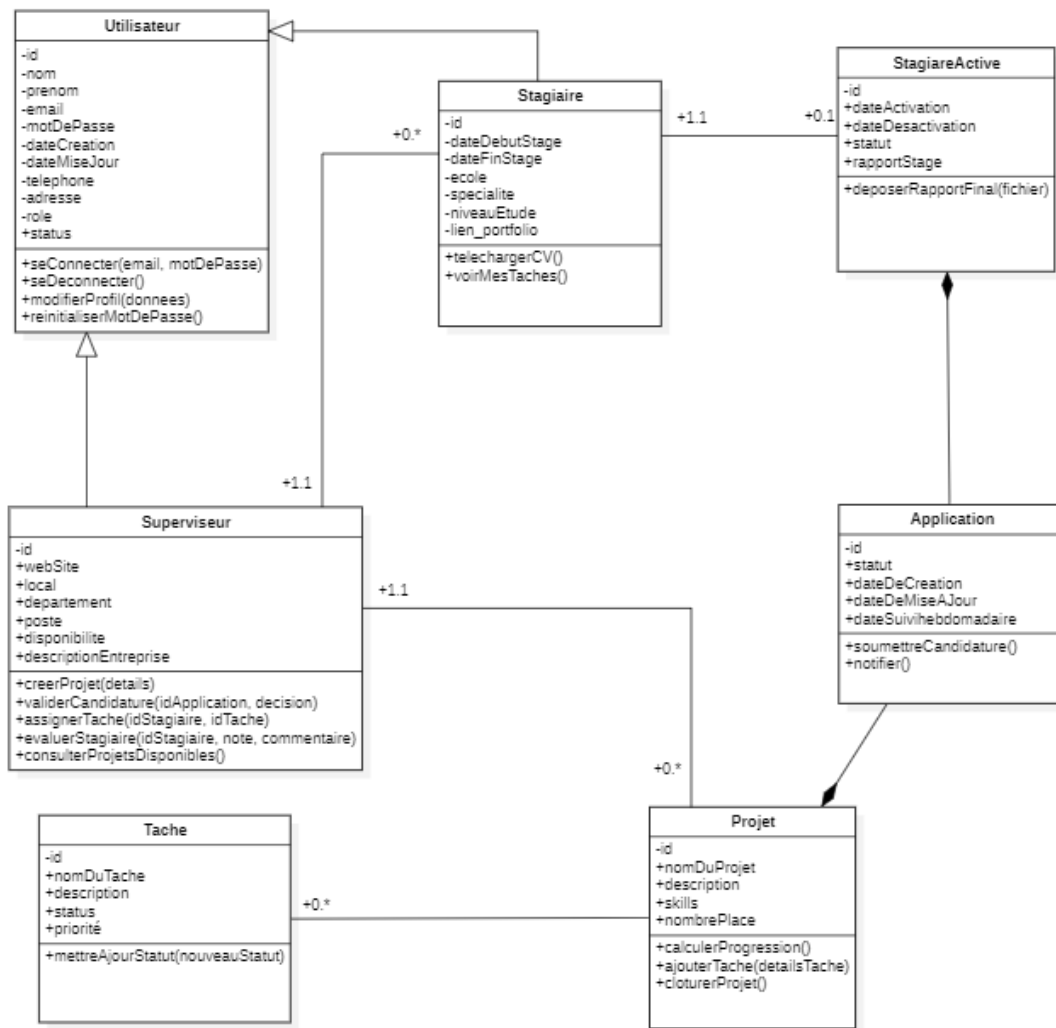


FIGURE 5.1 – Diagramme de classes UML : entités principales et relations du modèle de données

Applications et Stagiaires : L'entité **Application** gère le processus de candidature et d'affectation. Elle est liée à un stagiaire et permet de suivre l'état des candidatures, créant ainsi un historique complet du processus d'admission aux stages.

Stagiaires Actifs : L'entité **StagiaireActive** représente les stagiaires actuellement en stage. Elle est liée à un **Stagiaire** et à un **Projet**, et inclut les informations de suivi actif comme l'état du rapport de stage et la méthode de dépôt du rapport final.

Correspondance avec le schéma PostgreSQL Le diagramme UML se traduit directement en schéma relationnel PostgreSQL avec les correspondances suivantes :

- **Entités UML → Tables PostgreSQL** : Les classes **Utilisateur**, **Stagiaire**, **Superviseur**, **Projet**, **Tache**, **Application** et **StagiaireActive** deviennent des tables dans la base de données.
- **Attributs → Colonnes** : Les propriétés de chaque classe deviennent des colonnes avec des types de données appropriés (VARCHAR, TEXT, INTEGER, TIMESTAMP, BOOLEAN, ENUM).
- **Associations → Clés étrangères** : Les relations entre classes sont implémentées via des contraintes de clés étrangères garantissant l'intégrité référentielle.
- **Cardinalités → Contraintes** : Les multiplicités (1-1, 1-n, n-m) sont traduites en contraintes NOT NULL, UNIQUE et relations un-à-plusieurs.

- **Spécialisation → Héritage** : L'entité **Superviseur** spécialise **Utilisateur** pour représenter le rôle spécifique de superviseur dans le système.

Cette conception assure une cohérence parfaite entre le modèle conceptuel UML et l'implémentation physique dans PostgreSQL, facilitant ainsi la maintenance, l'évolution et la compréhension du système de données.

Chapitre 6

Architecture technique

6.1 Architecture globale

StageFlow adopte une architecture client-serveur en trois couches logiques : présentation, services applicatifs et persistance des données.

La couche présentation est assurée par le frontend React/Vite, qui fournit des interfaces réactives, modulaires et adaptées à chaque rôle utilisateur. La couche services est implémentée avec Node.js/Express et expose une API REST structurée par domaines fonctionnels (authentification, stages, tâches, rapports, administration). La couche données repose sur PostgreSQL, garantissant l'intégrité relationnelle et la cohérence transactionnelle des opérations critiques.

Cette organisation favorise une séparation claire des responsabilités, améliore la maintenabilité du code et facilite l'évolutivité de la plateforme. Les contrôleurs orchestrent la logique métier, les middlewares assurent les préoccupations transversales (authentification, validation, gestion d'erreurs, logging), et la base de données matérialise les règles d'intégrité via contraintes de clés étrangères, index adaptés et historisation des actions critiques.

6.2 Stack technologique

Le choix de la stack technologique a été guidé par des critères de performance, de productivité, de maintenabilité et de sécurité dans un contexte full-stack moderne.

Frontend : React et Vite permettent de construire une interface réactive, modulaire et rapide au chargement. Axios est utilisé pour la communication HTTP avec l'API backend, avec une gestion centralisée des en-têtes d'authentification JWT et une gestion cohérente des erreurs réseau.

Backend : Node.js et Express offrent un socle léger et efficace pour implémenter une API REST. Joi est utilisé pour la validation stricte et déclarative des données entrantes, bloquant les incohérences dès la couche d'entrée. JWT soutient l'authentification stateless et sécurisée, tandis que le modèle RBAC (Role-Based Access Control) encadre les autorisations selon le rôle de l'utilisateur connecté.

Base de données : PostgreSQL assure une persistance robuste et fiable des données métier. Son modèle relationnel facilite la traçabilité des workflows critiques liés aux stages, aux tâches, aux remarques et aux rapports. Les index ciblés et les contraintes d'intégrité optimisent les performances et garantissent la cohérence.

6.3 Sécurité et validation

La validation des données est implémentée via Joi au niveau des routes, bloquant les requêtes invalides avant traitement. L'authentification JWT protège l'accès aux ressources critiques de l'API, tandis que RBAC garantit qu'un utilisateur n'accède qu'aux opérations compatibles avec

son rôle métier. Les mots de passe sont hashés via bcrypt. Les logs des actions sensibles sont enregistrés pour l'auditabilité et l'investigation en cas d'incident.

6.3.1 Flux complet d'authentification JWT

Le système d'authentification JWT de StageFlow repose sur un flux stateless où le frontend collecte les identifiants, le backend valide la requête, signe un jeton court et les routes protégées vérifient ensuite ce jeton à chaque accès.

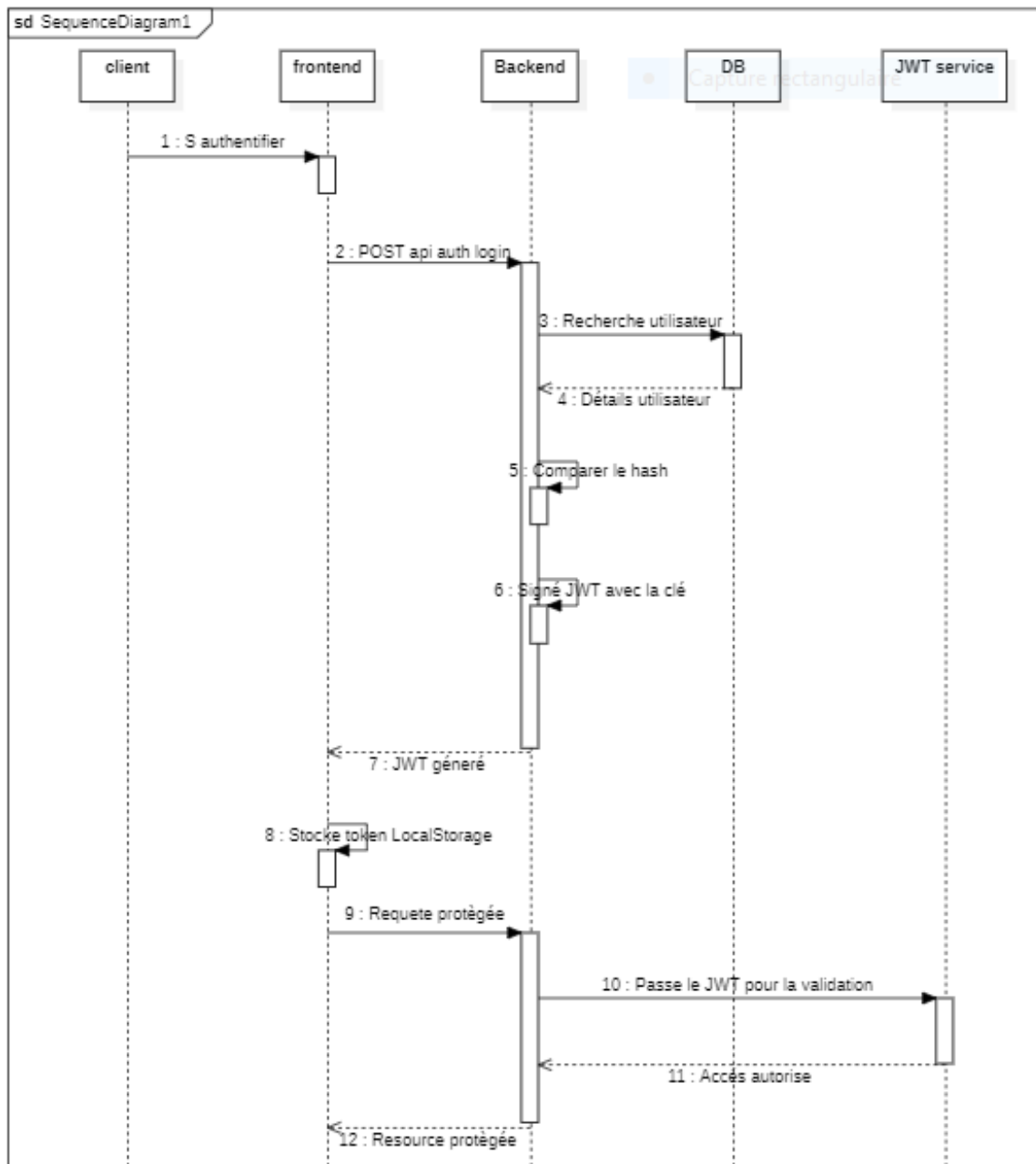


FIGURE 6.1 – Figure IV.3 : : Diagramme de séquence du flux d'authentification JWT

Séquence sécurisée

1. L'utilisateur saisit ses identifiants dans le formulaire de connexion, puis le frontend vérifie le format avant d'envoyer la requête.

2. Le backend valide les données reçues, compare le mot de passe avec le hash bcrypt stocké et refuse toute entrée incohérente.
3. Si l'authentification réussit, un JWT signé avec `process.env.JWT_SECRET` est généré avec l'identité et le rôle de l'utilisateur.
4. Le frontend stocke le token et l'ajoute ensuite dans l'en-tête `Authorization: Bearer ...` pour les routes protégées.
5. Le middleware JWT vérifie la signature, puis RBAC confirme que le rôle autorise l'accès ; sinon, une réponse 401 est renvoyée.
6. En cas d'expiration ou d'invalidité du jeton, le frontend supprime l'état local et redirige l'utilisateur vers la page de connexion.

Cette architecture renforce la sécurité en combinant hachage bcrypt, signature du jeton, expiration limitée, validation stricte des entrées et contrôle d'accès par rôles. Elle supprime la dépendance à une session serveur et garantit une authentification simple, traçable et adaptée à un environnement web distribué.

6.4 Fiabilité et qualité

La qualité de la plateforme repose sur la cohérence des workflows métier, la stabilité des composants techniques et la vérification continue du comportement applicatif. Les tests fonctionnels par rôle permettent de valider les scénarios critiques de bout en bout : création et gestion des stages, création des comptes étudiants, assignation et suivi des tâches, suivi de l'avancement et validation des rapports.

La robustesse est renforcée par une structuration claire du code (routes, contrôleurs, middlewares, validations), une gestion explicite et documentée des erreurs, et une modélisation relationnelle cohérente dans PostgreSQL. Des revues régulières des dépendances et des configurations de sécurité contribuent à limiter la dette technique et à maintenir un niveau de fiabilité compatible avec un usage réel en environnement académique et professionnel.

Chapitre 7

Interfaces utilisateur et captures d'écran

Ce chapitre est entièrement dédié aux captures d'écran de l'application StageFlow. Il contient les interfaces les plus importantes et représentatives du fonctionnement réel de la plateforme, afin d'illustrer concrètement les fonctionnalités principales implémentées.

Sélectionner uniquement les captures pertinentes : authentification, tableau de bord administrateur, gestion des stagiaires, création de stage, gestion des tâches, suivi de progression, espace étudiant, gestion des rapports et workflow d'encadrement. Éviter les captures répétitives ou secondaires afin de conserver une présentation professionnelle, lisible et cohérente avec un rapport de mémoire.

Chaque capture doit être accompagnée :

- d'un titre académique clair ;
- d'une courte description fonctionnelle ;
- d'une explication du rôle de l'interface dans le workflow global de StageFlow.

Les images seront insérées ci-dessous lorsque vous les fournirez. Pour l'instant chaque section contient une zone réservée (placeholder) indiquant l'image attendue et laissant la place pour l'insertions ultérieure.

7.1 Authentification — Écran de connexion

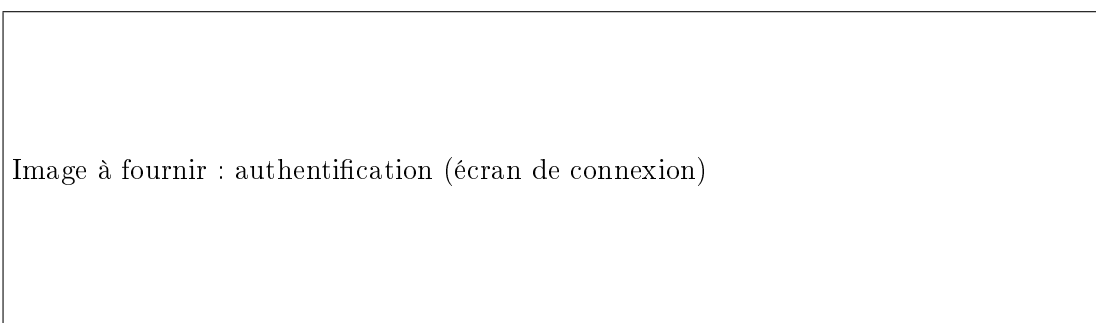


FIGURE 7.1 – Authentification — Écran de connexion (image fournie ultérieurement)

Description fonctionnelle Écran permettant à un utilisateur (Administrateur, Superviseur, Étudiant) de se connecter via identifiants protégés. Gère les erreurs d'authentification et les redirections selon le rôle.

Rôle dans le workflow Point d'entrée sécurisé de la plateforme ; contrôle l'accès aux ressources en appliquant le RBAC et initialise le contexte utilisateur pour les parcours suivants.

7.2 Tableau de bord administrateur



FIGURE 7.2 – Tableau de bord Administrateur (image fournie ultérieurement)

Description fonctionnelle Vue synthétique des indicateurs système : nombre d'utilisateurs, stages actifs, alertes et accès rapides aux fonctions d'administration.

Rôle dans le workflow Permet à l'administrateur de superviser l'état global du système, gérer les comptes et intervenir en cas d'incident ou d'anomalie.

7.3 Gestion des stagiaires



FIGURE 7.3 – Gestion des stagiaires — liste et fiche (image fournie ultérieurement)

Description fonctionnelle Interface listant les stagiaires, permettant la création/modification des fiches et l'affectation aux stages.

Rôle dans le workflow Outil central pour le superviseur : associer un étudiant à un stage, suivre ses informations et historique administratif.

7.4 Création de stage

Description fonctionnelle Formulaire de création d'un stage : titre, description, dates, objectifs pédagogiques et superviseur référent.

Image à fournir : création de stage

FIGURE 7.4 – Création de stage — formulaire de saisie (image fournie ultérieurement)

Rôle dans le workflow Permet d’enregistrer l’entité stage qui servira de conteneur aux tâches, au suivi et à la production du rapport final.

7.5 Gestion des tâches

Image à fournir : gestion des tâches

FIGURE 7.5 – Gestion des tâches — création et suivi (image fournie ultérieurement)

Description fonctionnelle Interface permettant la création, l’assignation, la mise à jour des statuts et la remise de livrables pour chaque tâche.

Rôle dans le workflow Outil opérationnel pour découper la mission en tâches, suivre l’avancement et consigner les retours de l’encadrement.

7.6 Suivi de progression

Image à fournir : suivi de progression

FIGURE 7.6 – Suivi de progression — indicateurs et timeline (image fournie ultérieurement)

Description fonctionnelle Vue consolidée des indicateurs de progression : tâches achevées, jalons atteints, et tendances temporelles.

Rôle dans le workflow Permet d'évaluer l'avancement global du stagiaire et d'anticiper les actions de remédiation si nécessaire.

7.7 Espace étudiant

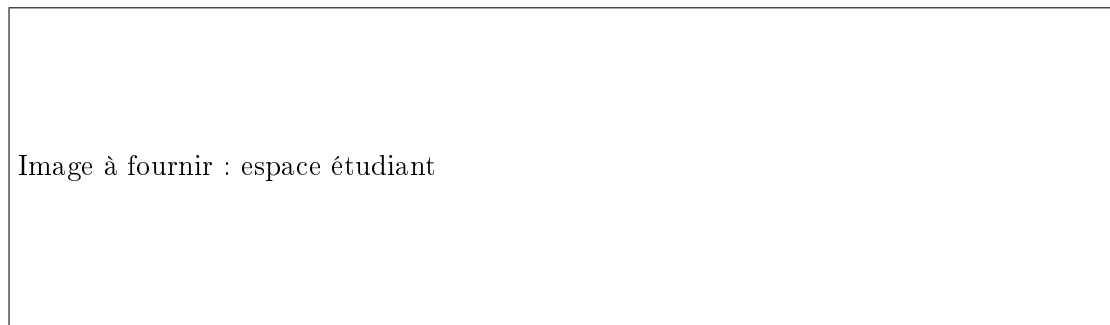


FIGURE 7.7 – Espace étudiant — tableau personnel et soumission de livrables (image fournie ultérieurement)

Description fonctionnelle Tableau de bord personnel de l'étudiant : tâches assignées, échéances, messages et accès au formulaire de rédaction/soumission du rapport.

Rôle dans le workflow Interface principale de l'étudiant pour exécuter le travail, soumettre des livrables et consulter les retours du superviseur.

7.8 Gestion des rapports



FIGURE 7.8 – Gestion des rapports — soumission et workflow de revue (image fournie ultérieurement)

Description fonctionnelle Interface de soumission et de revue du rapport de stage, avec historique des versions et commentaires du superviseur.

Rôle dans le workflow Conclut le cycle pédagogique : réception, revue et validation formelle du rapport.

7.9 Workflow d'encadrement (vue synthétique)

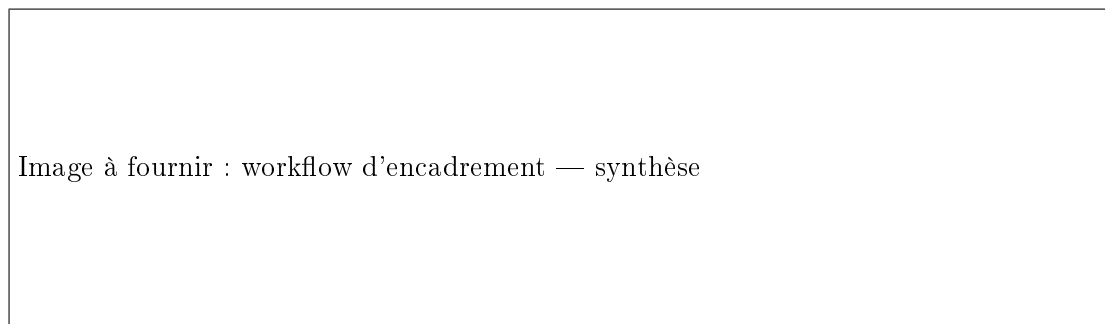


FIGURE 7.9 – Workflow d'encadrement — synthèse des interactions (image fournie ultérieurement)

Description fonctionnelle Vue synthétique montrant les interactions essentielles entre superviseur et étudiant autour des tâches, des retours et de la validation des livrables.

Rôle dans le workflow Fournit une vision d'ensemble utile pour les jurys et la documentation académique.

Chapitre 8

Outils et technologies utilisés

Ce chapitre présente les principaux outils et technologies employés dans le développement de StageFlow. Chaque section est centrée sur une description technique de l'outil, accompagnée d'une illustration représentative issue du dossier `screen/`.

8.1 Git et GitHub



FIGURE 8.1 – Git et GitHub — contrôle de version distribué

Description technique Git est un système de gestion de versions distribué permettant de suivre l'historique complet du code source, de gérer des branches et de fusionner des modifications de manière fiable. GitHub est une plateforme d'hébergement de dépôts Git qui facilite la collaboration, la revue de code et la conservation centralisée des versions du projet.

8.2 Node.js



FIGURE 8.2 – Node.js — runtime JavaScript côté serveur

Description technique Node.js est un environnement d'exécution JavaScript côté serveur basé sur le moteur V8. Il repose sur un modèle événementiel non bloquant, adapté aux applications web nécessitant de nombreuses opérations I/O asynchrones.

8.3 Express.js



FIGURE 8.3 – Express.js — framework web minimaliste

Description technique Express.js est un framework web léger pour Node.js qui fournit un système de routage, des middlewares réutilisables et une structure simple pour construire des API REST. Il simplifie le traitement des requêtes HTTP et l'organisation des points d'accès backend.

8.4 PostgreSQL



FIGURE 8.4 – PostgreSQL — système de gestion de base de données relationnelle

Description technique PostgreSQL est un système de gestion de base de données relationnelle open-source, connu pour sa robustesse, sa conformité au standard SQL et ses capacités transactionnelles. Il assure une persistance cohérente des données et un bon niveau d'intégrité référentielle.

8.5 Docker

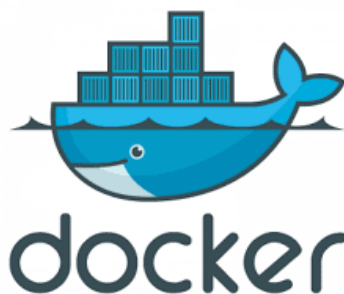


FIGURE 8.5 – Docker — conteneurisation et portabilité

Description technique Docker est une plateforme de conteneurisation qui encapsule une application et ses dépendances dans des conteneurs isolés. Elle permet d’obtenir des environnements reproductibles entre le développement, les tests et le déploiement.

8.6 JWT (JSON Web Token)



FIGURE 8.6 – JWT — authentification par jeton stateless

Description technique JWT est un standard de jeton signé qui permet de transporter des informations d’identité et d’autorisation de manière compacte et vérifiable. Il est couramment utilisé pour l’authentification stateless dans les applications web modernes.

Conclusion générale et perspectives

Le projet StageFlow constitue une contribution concrète à la digitalisation du suivi et de l'encadrement des stagiaires au sein des structures académiques et professionnelles. À travers une approche structurée et orientée workflow métier, ce projet a permis de concevoir et de développer une plateforme web multi-rôles dédiée à la gestion des stages, au suivi des tâches, à l'encadrement des stagiaires ainsi qu'à la gestion des rapports de fin de stage.

L'objectif principal de cette plateforme était de centraliser les interactions entre les différents acteurs du processus de stage tout en améliorant la traçabilité, l'organisation et la supervision des activités réalisées durant le cycle de stage. Contrairement aux plateformes publiques de recrutement, StageFlow adopte une logique interne de gestion et de suivi, où le superviseur joue un rôle central dans l'intégration, l'encadrement et l'évaluation du stagiaire.

L'une des principales forces du projet réside dans la séparation claire des responsabilités entre les trois rôles principaux : administrateur, superviseur et étudiant. Cette organisation permet de garantir une meilleure cohérence fonctionnelle, une gestion plus sécurisée des accès ainsi qu'un contrôle précis des opérations métier. Le superviseur peut créer les stages, gérer les comptes étudiants, attribuer des tâches et assurer le suivi de progression, tandis que l'étudiant dispose d'un espace dédié lui permettant de consulter ses missions, mettre à jour son avancement et soumettre ses rapports.

Sur le plan technique, la plateforme repose sur une architecture full-stack moderne utilisant React/Vite pour le frontend, Node.js/Express pour le backend et PostgreSQL pour la gestion des données. L'intégration de JWT pour l'authentification, du modèle RBAC pour la gestion des permissions ainsi que de Joi pour la validation des données a permis de renforcer la sécurité, la fiabilité et la maintenabilité de l'application. L'architecture MVC adoptée facilite également l'évolutivité du projet et la séparation des responsabilités techniques.

Le projet a également permis de mettre en pratique plusieurs concepts avancés liés au développement web moderne, notamment la conception d'API REST, la gestion sécurisée des utilisateurs, la structuration des workflows métier, la gestion documentaire et l'organisation des composants frontend. Cette expérience a représenté une opportunité importante pour approfondir les compétences en ingénierie logicielle, en conception d'architecture applicative et en développement full-stack.

Malgré les fonctionnalités déjà implémentées, plusieurs perspectives d'amélioration peuvent être envisagées afin d'enrichir davantage la plateforme et d'augmenter son niveau de robustesse et de professionnalisation.

Perspectives

- Renforcer la couverture des tests automatisés (tests unitaires, tests d'intégration et tests end-to-end) afin d'améliorer la qualité logicielle et la stabilité du système.
- Améliorer le système de notifications et de communication en temps réel entre superviseurs et stagiaires via WebSocket ou services temps réel avancés.
- Ajouter une gestion documentaire plus avancée avec historique des versions, audit des modifications et amélioration de la traçabilité des rapports.

- Optimiser davantage la sécurité applicative à travers un système de logs centralisés, de monitoring et de gestion avancée des incidents.
- Déployer la plateforme dans un environnement cloud scalable afin d'améliorer les performances, la disponibilité et la maintenance de l'application.
- Intégrer des fonctionnalités complémentaires telles que la génération automatique de rapports PDF, la gestion des évaluations détaillées ou encore le suivi statistique de progression des stages.
- Étendre l'architecture pour supporter ultérieurement plusieurs établissements, départements ou structures d'encadrement dans un environnement multi-organisation.

En conclusion, StageFlow représente une solution moderne et évolutive permettant de structurer efficacement le suivi des stages et l'encadrement des stagiaires dans un environnement numérique centralisé. Ce projet illustre l'importance des technologies web dans l'amélioration des processus de gestion académique et professionnelle, tout en mettant en avant les enjeux de sécurité, de traçabilité et de qualité logicielle dans le développement des applications modernes.

Bibliographie

- [1] Facebook Inc. (2024). *React : A JavaScript library for building user interfaces*. Documentation officielle. Disponible sur : <https://react.dev>
- [2] OpenJS Foundation (2024). *Node.js : JavaScript Runtime built on Chrome's V8 JavaScript engine*. Documentation officielle. Disponible sur : <https://nodejs.org/docs>
- [3] Express.js Contributors (2024). *Express.js - Fast, unopinionated web framework for Node.js*. Documentation officielle. Disponible sur : <https://expressjs.com>
- [4] PostgreSQL Global Development Group (2024). *PostgreSQL : The World's Most Advanced Open Source Relational Database*. Documentation officielle. Disponible sur : <https://www.postgresql.org/docs>
- [5] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force (IETF). Disponible sur : <https://tools.ietf.org/html/rfc7519>
- [6] Hapi.js Contributors (2024). *Joi : Data validation for JavaScript*. Documentation officielle. Disponible sur : <https://joi.dev>
- [7] Software Freedom Conservancy (2024). *Git : Fast Version Control System*. Documentation officielle. Disponible sur : <https://git-scm.com/doc>